

TD 2 — Authentification & Base de données Supabase

Durée : 2h · Travail en binôme · Pré-requis : TD1 terminé (projet Vite/React déployé sur Vercel)

 Objectifs du TD	
Intégrer Supabase	Connecter un backend PostgreSQL à votre app React
Authentification	Page de connexion (login/register) avec Supabase Auth
Dashboard protégé	Redirection automatique après connexion
Gestion utilisateurs	Lister et créer des utilisateurs depuis le dashboard

Partie A — Récapitulatif & préparation (15 min)

Rappel TD1

Avant de commencer, vérifiez : votre projet mon-kanban tourne sur `http://localhost:5173/`, votre dépôt GitHub est à jour (git status propre), et votre URL Vercel fonctionne encore.

A.1 Vérification de l'environnement

Ouvrez un terminal dans le dossier mon-kanban et lancez :

```
npm run dev
```

Vérifiez que la page s'affiche bien, puis contrôlez l'état Git :

```
git status
git log --oneline -5
```

A.2 Installation du SDK Supabase

Supabase fournit une bibliothèque JavaScript qui s'occupe de tout : connexion, authentification, requêtes BDD. Dans votre terminal (projet ouvert) :

```
npm install @supabase/supabase-js
```

Puis installez React Router pour la navigation entre pages :

```
npm install react-router-dom
```

Pourquoi React Router ?

Votre app aura plusieurs 'pages' : /login (page de connexion) et /dashboard (tableau de bord). React Router permet de naviguer entre elles sans rechargement — c'est une Single Page Application (SPA).

Partie B — Configurer Supabase (20 min)

B.1 Créer le projet Supabase

1. Connectez-vous sur <https://supabase.com/> avec votre compte GitHub (créé en TD1).
2. Cliquez sur New project.
3. Remplissez : Nom : kanban-rt | Mot de passe BDD : générez-en un fort et notez-le | Région : West EU (Ireland).
4. Cliquez Create new project. Attendez 1 à 2 minutes que la BDD soit prête.

B.2 Récupérer les clés API

Dans votre projet Supabase, allez dans Settings → API. Vous trouverez deux clés essentielles :

Clé	Rôle	Usage
Project URL	Adresse de votre BDD	VITE_SUPABASE_URL
anon / public	Clé publique côté client	VITE_SUPABASE_ANON_KEY

B.3 Fichier .env.local

À la racine de mon-kanban, créez le fichier .env.local (jamais commité sur GitHub) :

```
# .env.local — Ne pas commiter !
VITE_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Sécurité

.env.local est automatiquement ignoré par Git grâce au .gitignore créé par Vite. Ne copiez jamais vos clés dans votre code source ou dans un commit !

B.4 Initialiser le client Supabase

Créez le fichier src/lib/supabase.js :

```
// src/lib/supabase.js
import { createClient } from '@supabase/supabase-js';

// Ces variables viennent du fichier .env.local
```

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;  
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;  
  
export const supabase = createClient(supabaseUrl, supabaseKey);
```

✓ Test rapide

Dans n'importe quel composant React, importez { supabase } from './lib/supabase' et faites un console.log(supabase). Si vous voyez un objet dans la console du navigateur, la connexion est prête.

Partie C — Page de connexion (Login / Register) (30 min)

C.1 Structure des fichiers

Votre projet doit avoir l'arborescence suivante après ce TD :

```
src/  
  lib/  
    supabase.js      ← déjà créé  
  pages/  
    LoginPage.jsx     ← à créer  
    DashboardPage.jsx ← à créer  
  components/  
    UserTable.jsx     ← à créer (Partie D)  
  App.jsx             ← à modifier  
  main.jsx
```

C.2 Configurer le routeur dans App.jsx

Remplacez tout le contenu de src/App.jsx par :

```
// src/App.jsx  
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';  
import { useEffect, useState } from 'react';  
import { supabase } from '../lib/supabase';  
import LoginPage from '../pages/LoginPage';  
import DashboardPage from '../pages/DashboardPage';  
  
function App() {  
  const [session, setSession] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  // Écoute les changements de session (connexion/déconnexion)  
  useEffect(() => {  
    supabase.auth.getSession().then(({ data }) => {  
      setSession(data.session);  
      setLoading(false);  
    });  
    const { data: listener } = supabase.auth.onAuthStateChange(  
      (_event, session) => setSession(session)  
    );  
    return () => listener.subscription.unsubscribe();  
  }, []);  
  
  if (loading) return <div>Chargement...</div>;  
  
  return (  
    <BrowserRouter>  
      <Routes>
```

```

    <Route path='/login'
      element={session ? <Navigate to='/dashboard' /> : <LoginPage />} />
    <Route path='/dashboard'
      element={session ? <DashboardPage session={session} /> : <Navigate
to='/login' />} />
    <Route path='*' element={<Navigate to={session ? '/dashboard' :
'/login'} />} />
  </Routes>
</BrowserRouter>
);
}

export default App;

```

💡 Logique de protection

Si l'utilisateur n'est pas connecté (session === null) et tente d'aller sur /dashboard, il est automatiquement redirigé vers /login. C'est la base d'une route protégée.

C.3 Créer la page LoginPage.jsx

Créez le dossier src/pages/ puis le fichier src/pages/LoginPage.jsx :

```

// src/pages/LoginPage.jsx
import { useState } from 'react';
import { supabase } from '../lib/supabase';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isRegister, setIsRegister] = useState(false);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e) {
    e.preventDefault();
    setError('');
    setLoading(true);
    let result;
    if (isRegister) {
      result = await supabase.auth.signUp({ email, password });
    } else {
      result = await supabase.auth.signInWithPassword({ email, password });
    }
    if (result.error) setError(result.error.message);
    setLoading(false);
  }

  return (

```


Partie D — Dashboard & gestion utilisateurs (35 min)

D.1 Créer la page DashboardPage.jsx

Créez le fichier src/pages/DashboardPage.jsx :

```
// src/pages/DashboardPage.jsx
import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import UserTable from '../components/UserTable';

export default function DashboardPage({ session }) {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  async function fetchUsers() {
    const { data, error } = await supabase
      .from('profiles')
      .select('*')
      .order('created_at', { ascending: false });
    if (!error) setUsers(data || []);
    setLoading(false);
  }

  useEffect(() => { fetchUsers(); }, []);

  async function handleLogout() {
    await supabase.auth.signOut();
  }

  return (
    <div style={{ minHeight: '100vh', background: '#F8FAFC' }}>
      <header style={{ background: '#1A8C82', color: 'white',
        padding: '1rem 2rem', display: 'flex', justifyContent: 'space-between'
      }}>
        <h1>📌 KanbanRT — Dashboard</h1>
        <div>
          <span style={{ marginRight: '1rem' }}>{session.user.email}</span>
          <button onClick={handleLogout}
            style={{ background: 'white', color: '#1A8C82',
              border: 'none', padding: '0.5rem 1rem', borderRadius: '6px',
              cursor: 'pointer' }}>
            Déconnexion
          </button>
        </div>
      </header>
      <main style={{ padding: '2rem' }}>
        <h2>👤 Utilisateurs inscrits</h2>
        {loading ? <p>Chargement...</p>
```

```

        : <UserTable users={users} onRefresh={fetchUsers} />
      </main>
    </div>
  );
}

```

D.2 Créer la table profiles dans Supabase

Dans votre projet Supabase, allez dans Table Editor → New table. Créez la table profiles avec les colonnes suivantes :

Colonne	Type	Défaut	Note
id	uuid	gen_random_uuid()	Clé primaire (PK)
email	text	(vide)	Email de l'utilisateur
full_name	text	(vide)	Nom complet
role	text	'member'	admin / member
created_at	timestampz	now()	Date de création auto

⚠ Row Level Security (RLS)

Supabase active le RLS par défaut. Pour ce TD, allez dans Authentication → Politiques et désactivez temporairement le RLS sur la table profiles (disable RLS). En production, on le réactivera avec des politiques précises.

D.3 Créer le composant UserTable.jsx

Créez le dossier src/components/ puis le fichier src/components/UserTable.jsx :

```

// src/components/UserTable.jsx
import { useState } from 'react';
import { supabase } from '../lib/supabase';

export default function UserTable({ users, onRefresh }) {
  const [newEmail, setNewEmail] = useState('');
  const [newName, setNewName] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  async function handleCreate(e) {
    e.preventDefault();
    setError('');
    setLoading(true);
    const { error } = await supabase
      .from('profiles')
      .insert([{ email: newEmail, full_name: newName, role: 'member' }]);
  }
}

```



```

    if (error) { setError(error.message); }
    else { setNewEmail(''); setNewName(''); onRefresh(); }
    setLoading(false);
  }

  async function handleDelete(id) {
    if (!confirm('Supprimer cet utilisateur ?')) return;
    await supabase.from('profiles').delete().eq('id', id);
    onRefresh();
  }

  return (
    <div>
      { /* Formulaire de création */ }
      <form onSubmit={handleCreate} style={{ display: 'flex', gap: '0.5rem',
        marginBottom: '1.5rem', flexWrap: 'wrap' }}>
        <input placeholder='Email' type='email' value={newEmail}
          onChange={e => setNewEmail(e.target.value)} required
style={inputStyle} />
        <input placeholder='Nom complet' value={newName}
          onChange={e => setNewName(e.target.value)} style={inputStyle} />
        <button type='submit' disabled={loading} style={btnStyle}>
          {loading ? '...' : '+ Ajouter'}
        </button>
      </form>
      {error && <p style={{ color: 'red' }}>{error}</p>}

      { /* Tableau */ }
      <table style={{ width: '100%', borderCollapse: 'collapse' }}>
        <thead>
          <tr style={{ background: '#1A8C82', color: 'white' }}>
            <th style={thStyle}>Email</th>
            <th style={thStyle}>Nom</th>
            <th style={thStyle}>Rôle</th>
            <th style={thStyle}>Créé le</th>
            <th style={thStyle}>Actions</th>
          </tr>
        </thead>
        <tbody>
          {users.map((u, i) => (
            <tr key={u.id} style={{ background: i % 2 === 0 ? '#F8FAFC' :
'white' }}>
              <td style={tdStyle}>{u.email}</td>
              <td style={tdStyle}>{u.full_name || '-'}</td>
              <td style={tdStyle}>{u.role}</td>
              <td style={tdStyle}>{new
Date(u.created_at).toLocaleDateString('fr-FR')}</td>
              <td style={tdStyle}>
                <button onClick={() => handleDelete(u.id)}

```

```

                style={{ background: '#DC2626', color: 'white', border:
'none',
                padding: '0.25rem 0.75rem', borderRadius: '4px', cursor:
'pointer' }}>
                Supprimer
            </button>
        </td>
    </tr>
</tbody>
</table>
    {users.length === 0 && <p style={{ textAlign: 'center', color: '#94A3B8'
}}>
        Aucun utilisateur pour l'instant.</p>}
    </div>
);
}

const thStyle = { padding: '0.75rem 1rem', textAlign: 'left' };
const tdStyle = { padding: '0.75rem 1rem', borderBottom: '1px solid #E2E8F0' };
const inputStyle = { padding: '0.5rem 0.75rem', border: '1px solid #CBD5E1',
    borderRadius: '6px', fontSize: '0.9rem' };
const btnStyle = { padding: '0.5rem 1rem', background: '#1A8C82', color:
'white',
    border: 'none', borderRadius: '6px', cursor: 'pointer' };

```

Partie E — Test, commit & déploiement (20 min)

E.1 Tester en local

5. Lancez `npm run dev`.
6. Allez sur `http://localhost:5173/` → vous devez être redirigé vers `/login`.
7. Créez un compte avec votre e-mail étudiant. Vérifiez dans Supabase → Authentication → Users que l'utilisateur est bien créé.
8. Connectez-vous → vous devez arriver sur `/dashboard`.
9. Ajoutez un utilisateur dans le formulaire → il doit apparaître dans le tableau.
10. Supprimez-le → il disparaît.
11. Cliquez Déconnexion → retour sur `/login`.

E.2 Commit et push

```
git add .
git commit -m "feat: add Supabase auth and user dashboard"
git push origin main
```

E.3 Configurer les variables d'environnement sur Vercel

12. Sur `vercel.com/dashboard`, ouvrez votre projet `mon-kanban`.
13. Allez dans Settings → Environment Variables.
14. Ajoutez `VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY` avec vos valeurs.
15. Cliquez Redeploy → votre app déployée doit maintenant fonctionner en production.

Redeploy obligatoire

Les variables d'environnement Vercel ne sont prises en compte qu'au prochain déploiement. Après les avoir ajoutées, cliquez sur Redeploy depuis l'onglet Deployments.

Livrable de fin de séance

À déposer sur la plateforme de cours

1. L'URL de votre dépôt GitHub (doit contenir `LoginPage.jsx`, `DashboardPage.jsx`, `UserTable.jsx`)
2. L'URL Vercel de l'app déployée avec l'authentification fonctionnelle
3. Une capture d'écran du dashboard avec au moins un utilisateur affiché dans le tableau

Bonus — Pour aller plus loin

- Ajouter un champ 'rôle' (admin/member) dans le formulaire de création.
- Implémenter la modification d'un utilisateur (bouton Éditer → formulaire inline).
- Afficher un badge coloré selon le rôle de l'utilisateur.
- Ajouter une barre de recherche pour filtrer les utilisateurs par email ou nom.
- Activer le RLS sur la table profiles et créer une politique SELECT publique.